Assignment 4 solution

1. Q1

- **Testbench**

```
import ALSU_PACKAGE::*;     Could not f
module ALSU_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? Variables
  int error_count, correct_count;
  //? interface signals
  logic clk;
  logic rst;
  logic cin;
  logic serial_in;
  logic direction;
  logic red_op_A;
  logic red_op_B;
  logic bypass_A;
  logic bypass_B;
  logic signed [2:0] A;
  logic signed [2:0] B;
  logic [5:0] out;
  logic [15:0] leds;
  opcode_e opcode;
  //? golden model outputs
  logic signed [5:0] out_g;
  logic [15:0] leds_g;
  //? input registers
  logic cin_reg;
  logic serial_in_reg;
  logic direction_reg;
  logic red_op_A_reg;
  logic red_op_B_reg;
  logic bypass_A_reg;
  logic bypass_B_reg;
  logic signed [2:0] A_reg;
  logic signed [2:0] B_reg;
  opcode_e opcode_reg;
  //? class object
  ALSU_Rand_Var ALSU_obj;
  //? DUT instantiation
  ALSU #(
      .INPUT_PRIORITY(INPUT_PRIORITY),
      .FULL_ADDER(FULL_ADDER)
) ALSU_DUT (
    .clk(clk),
    .rst(rst),
    .cin(cin),
    .serial_in(serial_in),
    .direction(direction),
    .red_op_A(red_op_A),
    .red_op_B(red_op_B),
    .bypass_A(bypass_A),
    .bypass_B(bypass_B),
    .A(A),
    .B(B),
    .opcode(opcode),
    .out(out),
    .leds(leds)
);
//? CLK generation
initial begin
  clk = 0;
  forever begin
    #1 clk = ~clk;
    ALSU_obj.clk = clk;
  end
end
//? Sampling logic
always @(posedge rst or posedge bypass_A or posedge bypass_B)
  ALSU_obj.cvr_gp.stop();

always @(negedge rst or negedge bypass_A or negedge bypass_B)
  if(!rst && !bypass_A && !bypass_B) ALSU_obj.cvr_gp.start();

always @(posedge clk)
  ALSU_obj.cvr_gp.sample();
//? Driver block
initial begin
  //*initialization
  ALSU_obj = new();
  clk = 0;
  assign_class_properties();
  //! ALSU_1 TEST
  assert_reset();
  $display("ALSU_1 TEST time = %0t", $time);
  //! ALU_2 TEST
  ALSU_obj.ass3_constraints.constraint_mode(0);
  for (int i = 0; i < 100; i++) begin
    assert (ALSU_obj.randomize());
    run_test();
  end
  $display("ALSU_2 TEST time = %0t", $time);
```

```
    //! ALU_3 TEST
    ALSU_obj.constraint_mode(0);
    ALSU_obj.rst = 0;
    ALSU_obj.bypass_A = 0;
    ALSU_obj.bypass_B = 0;
    ALSU_obj.red_op_A = 0;
    ALSU_obj.red_op_B = 0;
    ALSU_obj.ass3_constraints.constraint_mode(1);
    for (int i = 0; i < 20; i++) begin
      assert (ALSU_obj.randomize());
      foreach (ALSU_obj.opcode_arr[i]) begin
        ALSU_obj.opcode = ALSU_obj.opcode_arr[i];
        run_test();
      end
    end
    //! Direct test for a certain opcode sequence
    ALSU_obj.opcode_arr = {OR, XOR, ADD, MULT, SHIFT, ROTATE};    Undefined variable: 'OR'.
    foreach (ALSU_obj.opcode_arr[i]) begin
        ALSU_obj.opcode = ALSU_obj.opcode_arr[i];
        run_test();
      end

    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end

  //? TASKS
    task assert_reset();
      rst = 1;
      #CLK_CYCLE;
      check_result(out_g, leds_g);
      rst = 0;
    endtask

    task check_result(input [5:0] out_exp, input [15:0] leds_exp);
      if (out_exp == out && leds_exp == leds) correct_count++;
      else begin
        error_count++;
          $display("xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx",$time);
          $display("out=%0b ,out_exp=%0b, leds=%0h, leds_exp=%0h", out, out_exp, leds, leds_exp);
      end
    endtask

    task assign_class_properties();
      rst = ALSU_obj.rst;
      A = ALSU_obj.A;
      B = ALSU_obj.B;
      cin = ALSU_obj.cin;
      opcode = ALSU_obj.opcode;
```

```
      serial_in = ALSU_obj.serial_in;
      direction = ALSU_obj.direction;
      red_op_A = ALSU_obj.red_op_A;
      red_op_B = ALSU_obj.red_op_B;
      bypass_A = ALSU_obj.bypass_A;
      bypass_B = ALSU_obj.bypass_B;
    endtask

    task run_test();
      assign_class_properties();
      #(CLK_CYCLE);
      check_result(out_g, leds_g);
    endtask
  //*GOLDEN MODEL
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      cin_reg <= 0;
      red_op_B_reg <= 0;
      red_op_A_reg <= 0;
      bypass_B_reg <= 0;
      bypass_A_reg <= 0;
      direction_reg <= 0;
      serial_in_reg <= 0;
      opcode_reg <= ADD;    Undefined variable: 'ADD'.
      A_reg <= 0;
      B_reg <= 0;
    end else begin
      cin_reg <= cin;
      red_op_B_reg <= red_op_B;
      red_op_A_reg <= red_op_A;
      bypass_B_reg <= bypass_B;
      bypass_A_reg <= bypass_A;
      direction_reg <= direction;
      serial_in_reg <= serial_in;
      opcode_reg <= opcode;
      A_reg <= A;
      B_reg <= B;
    end
  end
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      out_g  <= 0;
      leds_g <= 0;
    end else if (bypass_A_reg || bypass_B_reg) begin
      if (INPUT_PRIORITY == "A") begin
        if (bypass_A_reg) begin
          out_g  <= A_reg;
          leds_g <= 0;
        end else begin
```

```
188         end else begin
189           out_g  <= B_reg;
190           leds_g <= 0;
191         end
192       end else begin
193         if (bypass_B_reg) begin
194           out_g  <= B_reg;
195           leds_g <= 0;
196         end else begin
197           out_g  <= A_reg;
198           leds_g <= 0;
199         end
200       end
201     end else begin
202       case (opcode_reg)
203         ADD: begin    Undefined variable: 'ADD'.
204           if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
205             if (FULL_ADDER == "ON") begin
206               out_g  <= {{3{A_reg[2]}}, A_reg} + {{3{B_reg[2]}}, B_reg} + cin_reg;
207               leds_g <= 0;
208             end else begin
209               out_g  <= {{3{A_reg[2]}}, A_reg} + {{3{B_reg[2]}}, B_reg};
210               leds_g <= 0;
211             end
212           end else begin
213             out_g  <= 0;
214             leds_g <= ~leds_g;
215           end
216         end
217         MULT: begin    Undefined variable: 'MULT'.
218           if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
219             out_g  <= A_reg * B_reg;
220             leds_g <= 0;
221           end else begin
222             out_g  <= 0;
223             leds_g <= ~leds_g;
224           end
225         end
226         SHIFT: begin    Undefined variable: 'SHIFT'.
227           if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
228             if (direction_reg) out_g <= {out_g[4:0], serial_in_reg};
229             else out_g <= {serial_in_reg, out_g[5:1]};
230             leds_g <= 0;
231           end else begin
232             out_g  <= 0;
233             leds_g <= ~leds_g;
234           end
235         end
```

```
236         ROTATE: begin    Undefined variable: 'ROTATE'.
237           if (red_op_A_reg == 0 && red_op_B_reg == 0) begin
238             if (direction_reg) out_g <= {out_g[4:0], out_g[5]};
239             else out_g <= {out_g[0], out_g[5:1]};
240             leds_g <= 0;
241           end else begin
242             out_g  <= 0;
243             leds_g <= ~leds_g;
244           end
245         end
246         OR: begin    Undefined variable: 'OR'.
247           if (red_op_A_reg || red_op_B_reg) begin
248             if (INPUT_PRIORITY == "A") begin
249               if (red_op_A_reg) out_g <= |A_reg;
250               else out_g <= |B_reg;
251             end else begin
252               if (red_op_B_reg) out_g <= |B_reg;
253               else out_g <= |A_reg;
254             end
255           end else out_g <= A_reg | B_reg;
256           leds_g <= 0;
257         end
258         XOR: begin    Undefined variable: 'XOR'.
259           if (red_op_A_reg || red_op_B_reg) begin
260             if (INPUT_PRIORITY == "A") begin
261               if (red_op_A_reg) out_g <= ^A_reg;
262               else out_g <= ^B_reg;
263             end else begin
264               if (red_op_B_reg) out_g <= ^B_reg;
265               else out_g <= ^A_reg;
266             end
267           end else out_g <= A_reg ^ B_reg;
268           leds_g <= 0;
269         end
270         default: begin
271           out_g  <= 0;
272           leds_g <= ~leds_g;
273         end
274       endcase
275     end
276   end
277 endmodule
```

- **Package**

```
package ALSU_PACKAGE;
    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    parameter MAXPOS = 3'b011;
    parameter MAXNEG = 3'b100;
    parameter ZERO = 3'b000;
    typedef enum logic [2:0] { OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7 } opcode_e;
    class ALSU_Rand_Var;
        rand bit rst, cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
        rand opcode_e opcode;
        rand opcode_e opcode_arr[6];
        rand bit [2:0] A,B;
        bit clk;

        covergroup cvr_gp;
            A_cp: coverpoint A {
                bins A_data_0 = {0};
                bins A_data_max = {MAXPOS};
                bins A_data_min = {MAXNEG};
                bins A_data_default = default;
            }
            A_data_cp: coverpoint A iff (red_op_A) {
                bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100};
            }
            B_cp: coverpoint B {
                bins B_data_0 = {0};
                bins B_data_max = {MAXPOS};
                bins B_data_min = {MAXNEG};
                bins B_data_default = default;
            }
            B_data_cp: coverpoint B iff (red_op_B && !red_op_A){
                bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100};
            }
            opcode_cp: coverpoint opcode {
                bins Bins_shift[] = {SHIFT,ROTATE};
                bins Bins_arith[] = {ADD,MULT};
                bins Bins_bitwise[] = {OR,XOR};
                illegal_bins Bins_invalid = {INVALID_6,INVALID_7};
                bins Bins_trans = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
            }
            //? I am writing the cover points of the assignment 4 with different styles only for practicing
            cin_cp: coverpoint cin iff (opcode == ADD);
            direction_cp: coverpoint direction iff(opcode inside {SHIFT, ROTATE});
            serial_in_cp: coverpoint serial_in iff(opcode == SHIFT);
            red_opA_cp: coverpoint red_op_A {option.weight = 0;}
            red_opB_cp: coverpoint red_op_B {
                option.weight = 0;
                bins red_op_B0 = {0};
                bins red_op_B1 = {1};
            }
```

```
            ALU_cross_1_cp: cross A_cp, B_cp, opcode_cp {
                option.cross_auto_bin_max = 0;    option.cross_auto_bin_max = 0 setting is non-LRM compliant. No auto cross
                bins A_B_cross = binsof(A_cp) && binsof(B_cp) && binsof(opcode_cp.Bins_arith);
            }
            ALU_cross_2_cp: cross A_data_cp, B_cp iff(opcode inside {OR,XOR}){
                option.cross_auto_bin_max = 0;    option.cross_auto_bin_max = 0 setting is non-LRM compliant. No auto cross
                bins A_walking_cross_B = binsof(A_data_cp) && binsof(B_cp.B_data_0);
            }
            ALU_cross_3_cp: cross B_data_cp, A_cp, opcode_cp{
                option.cross_auto_bin_max = 0;    option.cross_auto_bin_max = 0 setting is non-LRM compliant. No auto cross
                bins B_walking_cross_A = binsof(B_data_cp) && binsof(A_cp.A_data_0) && binsof(opcode_cp) intersect {OR,XOR};
            }
            ALU_cross_4_cp: cross red_opA_cp, red_opB_cp iff (opcode != OR && opcode != XOR) {
                option.cross_auto_bin_max = 0;    option.cross_auto_bin_max = 0 setting is non-LRM compliant. No auto cross
                bins invalid_case = binsof(red_opA_cp) intersect {1} || binsof(red_opB_cp.red_op_B1);
            }
        endgroup

        function new();
            rst = 0;
            cin = 0;
            serial_in = 0;
            direction = 0;
            red_op_A = 0;
            red_op_B = 0;
            bypass_A = 0;
            bypass_B = 0;
            opcode = opcode_e'($random);
            A = $random;
            B = $random;
            cvr_gp = new();
        endfunction

        constraint ass2_constraints {
            rst dist {0:=99, 1:=1};
            bypass_A dist {0:=99, 1:=1};
            bypass_B dist {0:=99, 1:=1};
            opcode dist {INVALID_6:=10, INVALID_7:=10, [OR:ROTATE]:=90};
            if(opcode == OR || opcode == XOR){
                red_op_A dist {1:=30, 0:=70};
                red_op_B dist {1:=30, 0:=70};
                if(red_op_A) {
                    $onehot(A);
                    B == 0;
                } else if(red_op_B){
                    $onehot(B);
                    A == 0;
                }
            } else if(opcode == ADD || opcode == MULT) {
                A dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                B dist {MAXPOS:=80, MAXNEG:=80, ZERO:=80, {3'd1,3'd2,3'd5,3'd6,3'd7}:=20};
                red_op_A dist {1:=1, 0:=99};
```

```
                red_op_B dist {1:=1, 0:=99};
            } else {
                red_op_A dist {1:=1, 0:=99};
                red_op_B dist {1:=1, 0:=99};
            }
        }

        constraint ass3_constraints {
            //constraint valid scenario
            foreach (opcode_arr[i])
                opcode_arr[i] != INVALID_6 && opcode_arr[i] != INVALID_7;
            //constraint unique values
            foreach (opcode_arr[i])
                foreach (opcode_arr[j])
                    if (i < j) opcode_arr[i] != opcode_arr[j];
        }

    endclass
endpackage
```

- **Do file**

```
vlib work
vlog ALSU_PACKAGE.sv ALSU.v ALSU_tb.sv +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
coverage save ALSU.ucdb -onexit
log -r /*
run -all
coverage exclude -src ALSU.v -line 76 -code b
coverage exclude -du ALSU -togglenode {cin_reg[1]}
quit -sim
vcover report ALSU.ucdb -du=ALSU -details -annotate -all -output code_cover_rpt.txt
vcover report ALSU.ucdb -package=ALSU_PACKAGE -cvg -details -annotate -all -output fun_cover_rpt.txt
```

- **Simulation results**

```
# ** Warning: In instance '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp ' the 'option.per_instance' is set to '0' (false). Assignment to members 'weight', 'goal' and 'comment' of 'option' would not have any effect.
# ALSU_1 TEST time = 2
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_6. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 1.
#    Time: 23 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_7. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 2.
#    Time: 27 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_6. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 3.
#    Time: 31 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_6. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 4.
#    Time: 71 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_7. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 5.
#    Time: 95 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_7. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 6.
#    Time: 121 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_7. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 7.
#    Time: 123 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_6. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 8.
#    Time: 151 ns  Iteration: 1  Instance: /ALSU_tb
# ** Error: (vsim-8565) Illegal state bin was hit at value=INVALID_6. The bin counter for the illegal bin '\/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp .opcode_cp.Bins_invalid' is 9.
#    Time: 173 ns  Iteration: 1  Instance: /ALSU_tb
# ALSU_2 TEST time = 202
# error_counts = 0, correct_counts = 227
```

- **Functional coverage**

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/ALSU_PACKAGE/ALSU_Rand_Var		100.00%				
TYPE cvr_gp		100.00%	100	100.00...		✓
CVP cvr_gp::A_cp		100.00%	100	100.00...		✓
CVP cvr_gp::A_data_cp		100.00%	100	100.00...		✓
CVP cvr_gp::B_cp		100.00%	100	100.00...		✓
CVP cvr_gp::B_data_cp		100.00%	100	100.00...		✓
CVP cvr_gp::opcode_cp		100.00%	100	100.00...		✓
CVP cvr_gp::cin_cp		100.00%	100	100.00...		✓
CVP cvr_gp::direction_cp		100.00%	100	100.00...		✓
CVP cvr_gp::serial_in_cp		100.00%	100	100.00...		✓
CVP cvr_gp::red_opA_cp		0.00%	100	0.00%		✓
CVP cvr_gp::red_opB_cp		0.00%	100	0.00%		✓
CROSS cvr_gp::ALU_cross_1_cp		100.00%	100	100.00...		✓
CROSS cvr_gp::ALU_cross_2_cp		100.00%	100	100.00...		✓
CROSS cvr_gp::ALU_cross_3_cp		100.00%	100	100.00...		✓
CROSS cvr_gp::ALU_cross_4_cp		100.00%	100	100.00...		✓
INST \/ALSU_PACKAGE::ALSU_Rand_Var::cvr_gp		100.00%	100	100.00...		✓
CVP A_cp		100.00%	100	100.00...		✓
CVP A_data_cp		100.00%	100	100.00...		✓
CVP B_cp		100.00%	100	100.00...		✓
CVP B_data_cp		100.00%	100	100.00...		✓
CVP opcode_cp		100.00%	100	100.00...		✓
CVP cin_cp		100.00%	100	100.00...		✓
CVP direction_cp		100.00%	100	100.00...		✓
CVP serial_in_cp		100.00%	100	100.00...		✓
CVP red_opA_cp		100.00%	100	100.00...		✓
CVP red_opB_cp		100.00%	100	100.00...		✓
CROSS ALU_cross_1_cp		100.00%	100	100.00...		✓
CROSS ALU_cross_2_cp		100.00%	100	100.00...		✓
CROSS ALU_cross_3_cp		100.00%	100	100.00...		✓
CROSS ALU_cross_4_cp		100.00%	100	100.00...		✓

- ## Code coverage

❖ Branch coverage

```
================================================================================
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%

================================Branch Details================================
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        118       118         0   100.00%

================================Toggle Details================================

Toggle Coverage for Design Unit work.ALSU

                                    Node      1H->0L      0L->1H                "Coverage"
                                    ----------------------------------------------------------
                                   A[0-2]          1           1                    100.00
                               A_reg[0-2]          1           1                    100.00
                                   B[0-2]          1           1                    100.00
                               B_reg[0-2]          1           1                    100.00
                                  bypass_A          1           1                    100.00
                              bypass_A_reg          1           1                    100.00
                                  bypass_B          1           1                    100.00
                              bypass_B_reg          1           1                    100.00
                                       cin          1           1                    100.00
                               cin_reg[0]          1           1                    100.00
                                       clk          1           1                    100.00
                                 direction          1           1                    100.00
                             direction_reg          1           1                    100.00
                                   invalid          1           1                    100.00
                            invalid_opcode          1           1                    100.00
                            invalid_red_op          1           1                    100.00
                                leds[0-15]          1           1                    100.00
                               opcode[0-2]          1           1                    100.00
                           opcode_reg[0-2]          1           1                    100.00
                                  out[0-5]          1           1                    100.00
                                  red_op_A          1           1                    100.00
                              red_op_A_reg          1           1                    100.00
                                  red_op_B          1           1                    100.00
                              red_op_B_reg          1           1                    100.00
                                       rst          1           1                    100.00
                                 serial_in          1           1                    100.00
                             serial_in_reg          1           1                    100.00

Total Node Count     =         59
Toggled Node Count   =         59
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (118 of 118 bins)


Total Coverage By Design Unit (filtered view): 100.00%
```

2. Q2

```
module general_sva(clk, a, b, c, d, y, valid);
    input bit clk, a, b, c, d, y, valid;

    assertion_1: assert property (@(posedge clk) a |-> ##2 b);
    assertion_2: assert property (@(posedge clk) a&&b |-> ##[1:3] c);

    sequence s11b;
        ##2 (!b);
    endsequence

    property prop1;
        @(posedge clk) $one_hot(y);
    endproperty

    property prop2;
        @(posedge clk) d==0 |=> !valid;
    endproperty

endmodule
```

3. Q3

- **Design**

```
module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
parameter WIDTH = 4;
input clk;
input rst_n;
input load_n;
input up_down;
input ce;
input [WIDTH-1:0] data_load;
output reg [WIDTH-1:0] count_out;
output max_count;
output zero;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        count_out <= 0;
    else if (!load_n)
        count_out <= data_load;
    else if (ce)
        if (up_down)
            count_out <= count_out + 1;
        else
            count_out <= count_out - 1;
end

assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
assign zero = (count_out == 0)? 1:0;

endmodule
```

- **Testbench**

```
import COUNTER_PACKAGE::*;    Could not find the
module counter_tb (count_if.TEST intf);
  //? class object
  counter_var count_rand;
  //? Driver block
  initial begin
    //* initialization
    count_rand = new();
    intf.rst_n = 1;
    intf.load_n = 1;
    intf.up_down = 0;
    intf.ce = 0;
    intf.data_load = $random;
    //! RESET the design
    @(negedge intf.clk) intf.rst_n = 0;
    @(negedge intf.clk) intf.rst_n = 1;
    //! constrained randomization
    for (int i = 0; i<100; i++) begin
        assert(count_rand.randomize());
        intf.data_load = count_rand.data_load;
        intf.ce = count_rand.ce;
        intf.up_down = count_rand.up_down;
        intf.rst_n = count_rand.rst_n;
        intf.load_n = count_rand.load_n;
        @(negedge intf.clk);
    end
    $stop;
  end
endmodule
```

- **Package**

```
package COUNTER_PACKAGE;
    parameter WIDTH = 4;
    class counter_var;
        rand bit rst_n, load_n, up_down, ce;
        rand bit [WIDTH-1:0] data_load;
        constraint c_dist {
            rst_n  dist {1:=95, 0:=5};
            load_n dist {0:=70, 1:=30};
            ce     dist {0:=30, 1:=70};
        }
    endclass
endpackage
```

- **Counter interface**

```
interface count_if (clk);
    parameter WIDTH = 4;
    input bit clk;
    logic rst_n, load_n, up_down, ce;
    logic [WIDTH-1:0] data_load, count_out;
    logic max_count, zero;

    modport TEST (
        input clk, count_out, max_count, zero,
        output rst_n, load_n, up_down, ce, data_load
    );

    modport DUT (
        input clk, rst_n, load_n, up_down, ce, data_load,
        output count_out, max_count, zero
    );

    modport MONITOR (
        input clk, rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero
    );
endinterface
```

- **Wrapper**

```
module counter_wrapper(count_if.DUT intf);

    counter #(.WIDTH(intf.WIDTH)) u_counter (
        .clk (intf.clk),
        .rst_n (intf.rst_n),
        .load_n (intf.load_n),
        .up_down (intf.up_down),
        .ce (intf.ce),
        .data_load (intf.data_load),
        .count_out (intf.count_out),
        .max_count (intf.max_count),
        .zero (intf.zero)
    );

endmodule
```

- **Top module**

```
module TOP();
    bit clk;
    always #1 clk = ~clk;
    count_if intf(clk);
    counter_wrapper count_wrap (intf);
    counter_tb TB(intf.TEST);
    bind counter_wrapper counter_sva counter_sva_inst(intf);
endmodule
```

- **SVA file**

```
module counter_sva(count_if.MONITOR intf);
    //? asynchronous reset assertion
    always_comb begin
        if(!intf.rst_n)
            arst_sva: assert final(intf.count_out == 0 && intf.max_count == 0 && intf.zero == 1);
    end
    //? PROPERTIES
    property zero_sva_check;
        (@(posedge intf.clk) intf.count_out == 0 |-> intf.zero);
    endproperty
    property max_out_sva_check;
        (@(posedge intf.clk) intf.count_out == '1 |-> intf.max_count);
    endproperty
    property load_sva_check;
        @(posedge intf.clk) disable iff(!intf.rst_n) !intf.load_n |=> intf.count_out == $past(intf.data_load);
    endproperty
    property ce_load_off_sva_check;
        @(posedge intf.clk) disable iff(!intf.rst_n) !intf.ce && intf.load_n |=> intf.count_out == $past(intf.count_out);
    endproperty
    property up_sva_check;
        @(posedge intf.clk) disable iff(!intf.rst_n) intf.ce && intf.load_n && intf.up_down |=> intf.count_out == $past(intf.count_out) + 1'b1;
    endproperty
    property down_sva_check;
        @(posedge intf.clk) disable iff(!intf.rst_n) intf.ce && intf.load_n && !intf.up_down |=> intf.count_out == $past(intf.count_out) - 1'b1;
    endproperty
    //? ASSERTIONS
    zero_sva:        assert property (zero_sva_check);
    max_out_sva:     assert property (max_out_sva_check);
    load_sva:        assert property (load_sva_check) ;
    ce_load_off_sva: assert property (ce_load_off_sva_check);
    up_sva:          assert property (up_sva_check);
    down_sva:        assert property (down_sva_check);
    //? COVER PROPERTIES
    zero_sva_cp:        cover property (zero_sva_check);
    max_out_sva_cp:     cover property (max_out_sva_check);
    load_sva_cp:        cover property (load_sva_check) ;
    ce_load_off_sva_cp: cover property (ce_load_off_sva_check);
    up_sva_cp:          cover property (up_sva_check);
    down_sva_cp:        cover property (down_sva_check);

endmodule
```

- ## DO file

```
vlib work
vlog counter.v COUNTER_PACKAGE.sv count_if.sv counter_tb.sv counter_sva.sv counter_wrapper.sv TOP.sv +cover -covercells
vsim -voptargs=+acc work.TOP -cover
coverage save counter.ucdb -onexit
#do wave.do
log -r /*
run -all
quit -sim
vcover report counter.ucdb -du=counter -details -annotate -all -output code_cover_rpt.txt
vcover report counter.ucdb -assert -directive -du=counter_sva -details -output sva_cover_rpt.txt
#vcover report counter.ucdb -package=COUNTER_PACKAGE -cvg -details -annotate -all -output fun_cover_rpt.txt
```

- ## Verification plan

	Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
2	COUNTER_1	assert reset signal and check the count_out bits to be low	direct test in the beginnig of the code then randomization under constraints to make reset signal be deasserted most of the time	-	immediate assertion for the asynchronous reset functionality **(arst_sva)**
3	COUNTER_2	when count_out bits are low check for zero signal to be asserted	randomization without constraints.	-	concurrent assertion **(zero_sva)**
4	COUNTER_3	when count_out bits are high check for max_out signal to be asserted	randomization without constraints.	-	concurrent assertion **(max_out_sva)**
5	COUNTER_4	when load is asserted, check the	randomization under constraints of the load signal to be active 70% of the	-	concurrent assertion **(load_sva)**
6	COUNTER_6	make sure that counter stop counting when the ce and load are off	randomization under constraints of the ce signal to be active 70% of the time	-	concurrent assertion **(ce_load_off_sva)**
7	COUNTER_7	check counting procedure for the different values of the up_down signal	while then enabled to make sure that the counter will make a full round up or down counting.	-	concurrent assertion **(up_sva) & (down_sva)**

- ## SVA coverage

```
Coverage Report by DU with details

=================================================================================
=== Design Unit: work.counter_sva
=================================================================================

Assertion Coverage:
    Assertions                       7         7         0   100.00%
----------------------------------------------------------------------
Name                 File(Line)                    Failure      Pass
                                                   Count        Count
----------------------------------------------------------------------
work.counter_sva::arst_sva
                     counter_sva.sv(5)                   0          1
work.counter_sva::zero_sva
                     counter_sva.sv(27)                  0          1
work.counter_sva::max_out_sva
                     counter_sva.sv(28)                  0          1
work.counter_sva::load_sva
                     counter_sva.sv(29)                  0          1
work.counter_sva::ce_load_off_sva
                     counter_sva.sv(30)                  0          1
work.counter_sva::up_sva
                     counter_sva.sv(31)                  0          1
work.counter_sva::down_sva
                     counter_sva.sv(32)                  0          1

Directive Coverage:
    Directives                       6         6         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                  Design Design   Lang File(Line)      Hits Status
                                      Unit   UnitType
--------------------------------------------------------------------------------------------
work.counter_sva::zero_sva_cp         counter_sva Verilog  SVA  counter_sva.sv(34)
                                                                                 12 Covered
work.counter_sva::max_out_sva_cp      counter_sva Verilog  SVA  counter_sva.sv(35)
                                                                                  5 Covered
work.counter_sva::load_sva_cp         counter_sva Verilog  SVA  counter_sva.sv(36)
                                                                                 29 Covered
work.counter_sva::ce_load_off_sva_cp  counter_sva Verilog  SVA  counter_sva.sv(37)
                                                                                  4 Covered
work.counter_sva::up_sva_cp           counter_sva Verilog  SVA  counter_sva.sv(38)
                                                                                  4 Covered
work.counter_sva::down_sva_cp         counter_sva Verilog  SVA  counter_sva.sv(39)
                                                                                  4 Covered
```

Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Includ
/TOP/count_wrap/counter_sva_inst/zero_sva_cp	SVA	✓	Off	20	1	Unli...	1	100%		✓
/TOP/count_wrap/counter_sva_inst/max_out_sva_cp	SVA	✓	Off	11	1	Unli...	1	100%		✓
/TOP/count_wrap/counter_sva_inst/load_sva_cp	SVA	✓	Off	60	1	Unli...	1	100%		✓
/TOP/count_wrap/counter_sva_inst/ce_load_off_sva_cp	SVA	✓	Off	11	1	Unli...	1	100%		✓
/TOP/count_wrap/counter_sva_inst/up_sva_cp	SVA	✓	Off	9	1	Unli...	1	100%		✓
/TOP/count_wrap/counter_sva_inst/down_sva_cp	SVA	✓	Off	9	1	Unli...	1	100%		✓

- **Code coverage**

```
=================================================================================
=== Design Unit: work.counter
=================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================

Toggle Coverage for Design Unit work.counter

                                    Node      1H->0L      0L->1H  "Coverage"
                                    ----------------------------------------
                                      ce           1           1      100.00
                                     clk           1           1      100.00
                            count_out[0-3]         1           1      100.00
                            data_load[0-3]         1           1      100.00
                                   load_n          1           1      100.00
                                max_count          1           1      100.00
                                    rst_n          1           1      100.00
                                  up_down          1           1      100.00
                                     zero          1           1      100.00

 Total Node Count     =         15
 Toggled Node Count   =         15
 Untoggled Node Count =          0

 Toggle Coverage      =     100.00% (30 of 30 bins)


 Total Coverage By Design Unit (filtered view): 100.00%
```

4. Q4

- **Verification plan**

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
REG_1	assert the reset signal and check the default reset value for each register	direct test using assert reset task	-	compare with a golden model if any of the regs captures a value different than the default reset value then it is a bug
REG_2	check writing ability in case when the write signal is disabled	direct test by applying the same data_in and looping on all possible addresses while write is disabled	-	compare with a golden model if any of the regs captures the data_in value then it is a bug
REG_3	check if sequential writing is working properly or not	direct test by applying the same data_in and looping on all possible addresses while write is enabled	-	compare with a golden model if any of the registers captured the input value before the next posedge of the clk then it is a bug
REG_4	check normal write operation for different data_in values	direct test by applying all ones and walking ones for each reg while other regs are set to zero, at last some random stimulus	-	compare with a golden model if the desiered reg sampled any different value from the input one then it is a bug also if any of the non-desierd regs changes its value also it is a bug
REG_5	check that data_out is reading compinationally not sequentially	direct test by looping on all possible addresses and check the data_out value	-	compare with a golden model to check if the data_out bus sampled the desiered value correct or not also check that it samples when the address pointer changes directly as if it is sampling synchronously with the clk then it is a bug

- **Testbench**

```
module config_reg_tb ();
  //? local parameters
  localparam MAX = 16'hffff;
  localparam ZERO = 0;
  localparam CLK_CYCLE = 20;
  //? Interface signals
  bit clk;
  logic reset;
  logic write;
  logic [15:0] data_in;
  logic [15:0] data_out;
  typedef enum logic [2:0] {
    adc0_reg,
    adc1_reg,
    temp_sensor0_reg,
    temp_sensor1_reg,
    analog_test,
    digital_test,
    amp_gain,
    digital_config
  } config_reg_e;
  config_reg_e reg_addr;
  //? Variables
  int error_count, correct_count;
  logic [15:0] reset_assoc[config_reg_e];
  logic [15:0] golden_reg [config_reg_e];
  //? DUT instantiation
  config_reg DUT (
      .clk(clk),
      .reset(reset),
      .write(write),
      .data_in(data_in),
      .address(reg_addr),
      .data_out(data_out)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? assign default reset values
  initial begin
    reset_assoc[adc0_reg]         = 16'hFFFF;
    reset_assoc[adc1_reg]         = 0;
    reset_assoc[temp_sensor0_reg] = 0;
    reset_assoc[temp_sensor1_reg] = 0;
    reset_assoc[analog_test]      = 16'hABCD;
    reset_assoc[digital_test]     = 0;
    reset_assoc[amp_gain]         = 0;
    reset_assoc[digital_config]   = 1;
  end
```

```
//? Driver block
initial begin
  //* initialization
  reset = 0;
  write = 0;
  data_in = 0;
  reg_addr = adc0_reg;
  #CLK_CYCLE;

  //! REG_1 TEST
  assert_reset();


  //! REG_2 TEST
  $display("\n******write data when write signal is disabled******");
  @(negedge clk);
  data_in = 16'hAAAA;
  reg_addr = reg_addr.first;
  for(int i=0; i<reg_addr.num; i++) begin
    @(negedge clk);
    check_result(reg_addr, golden_reg);
    reg_addr = reg_addr.next;
  end


  //! REG_3 TEST
  $display("\n*******check that write operation is completely sequential not compinational********");
  @(negedge clk);
  //*check if any register sampled the input direct before the next positive edge
  write = 1;
  check_result(reg_addr, golden_reg);


  //! REG_4 TEST
  $display("\n*******check normal write operations with different input values********");
  //* fill regfile with zeros
  $display("\n----fill regfile with zeros----");
  fill(reg_addr, data_in, 0);
  check_result(reg_addr, golden_reg);


  //* check writing all ones in each reg while other regs are zeros
  $display("\n----writing all ones in each register while reseting others----");
  data_in = 16'hFFFF;
  reg_addr = reg_addr.first;
```

```
  for(int i=0; i<reg_addr.num; i++) begin
    $display("16'hFFFF written in address: %s while other addresses are zero", reg_addr.name());
    golden_reg[reg_addr] = data_in;
    @(negedge clk);
    check_result(reg_addr, golden_reg);
    reg_addr = reg_addr.next;
    fill(reg_addr, data_in, 0);
  end


  //* check writing walking ones in each register while other regs are zero
  $display("\n----writing walking ones in each register----");
  reg_addr = reg_addr.first;
  for(int i=0; i<reg_addr.num; i++) begin
    data_in = 16'h0001;
    for(int j=0; j<16; j++) begin
      golden_reg[reg_addr] = data_in;
      @(negedge clk);
      $display("data_in = %b written in address: %s while other addresses are zero", data_in ,reg_addr.name());
      check_result(reg_addr, golden_reg);
      fill(reg_addr, data_in, 0);
      data_in = data_in << 1;
    end
    reg_addr = reg_addr.next;
  end


  //* randomization
  $display("\n******randomization phase******");
  reg_addr = reg_addr.first;
  for(int i=0; i<reg_addr.num; i++) begin
    for(int j=0; j<100; j++) begin
      data_in = $random;
      write = $random;
      if(write) golden_reg[reg_addr] = data_in;
      @(negedge clk);
      if (write) $display("data_in = %b not written in address: %s as write is disabled", data_in, reg_addr.name());
      else $display("data_in = %b written in address: %s while other addresses are zero", data_in ,reg_addr.name());
      check_result(reg_addr, golden_reg);
    end
    reg_addr = reg_addr.next;
  end


  $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
  $stop;
end
```

```
//? TASKS
task assert_reset();
  reset = 1;
  @(negedge clk);
  $display("\n*****RESET PHASE*****");
  check_result(reg_addr, reset_assoc);
  reset = 0;
endtask

task fill(config_reg_e reg_address, logic [15:0] data_input, fill_value);
  data_in = fill_value;
  reg_addr = reg_addr.first;
  for(int i=0; i<reg_addr.num; i++) begin
    @(negedge clk);
    golden_reg[reg_addr] = data_in;
    reg_addr = reg_addr.next;
  end
  reg_addr = reg_address;
  data_in = data_input;
endtask

task automatic check_result(config_reg_e reg_address, ref logic [15:0] gold_reg[config_reg_e]);
  reg_addr = reg_addr.first;

  for (int i = 0; i < reg_addr.num; i++) begin
    #(CLK_CYCLE / 20);

    if (data_out == gold_reg[reg_addr]) correct_count++;
    else begin
      error_count++;
      $display(
          "\txxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx",
          $time);
      $display("\taddr=%s(%0d) actual_data_out=%b expected_data_out=%b\n", reg_addr.name(),
               int'(reg_addr), data_out, gold_reg[reg_addr]);
    end

    golden_reg[reg_addr] = data_out;
    reg_addr = reg_addr.next;
  end
  reg_addr = reg_address;
endtask

endmodule
```

- **Bug report**
  - **Bug number 1**
    - **Design input for bug to appear:**

      Assert reset to be 1
    - **Expected behavior**

      Analog_test register is reset to 16'hABCD
    - **Observed behavior**

      Analog_test register is reset to 16'hABCC
  - **Bug number 2**
    - **Design input for bug to appear:**

      Write zero to register adc0_reg
    - **Expected behavior**

      adc0_reg samples value zero

- **Observed behavior**

  Most significant bit in the adc0_reg stuck at one and the sampled value was 16'h8000

- **Bug number 3**
  - **Design input for bug to appear:**

    Write 16'hFF00 to register adc_reg

  - **Expected behavior**

    adc1_reg samples the input value correctly

  - **Observed behavior**

    adc1_reg swaps the least significant 4-bits with the most significant 4-bits and stores the value 16'h00FF instead of 16'hFF00

- **Bug number 4**
  - **Design input for bug to appear:**

    write 1 to register temp_sensor0_reg

  - **Expected behavior**

    Temp_sensor0_reg samples the value of the input correctly 1

  - **Observed behavior**

    Temp_sensor0_reg stores the input value after shifting it left by 1-bit

- **Bug number 5&6**
  - **Design input for bug to appear:**

    Write 16'hAAAA to digital_test register and 16'hFFFF to amp_gain register

  - **Expected behavior**

    Each of the previous registers samples the values correctly

  - **Observed behavior**

    The value that is required to be stored in the digital_test register is written in the amp_gain register instead and also the value sent to the amp_gain register is written in the digital_test register, so the two values swapped between the two registers.

- **Bug number 7**
  - **Design input for bug to appear:**

    Write 16'FFFF to digital_config register

- **Expected behavior**

  digital_config samples the input data value correctly

- **Observed behavior**

  The most significant bit in the digital_config register stuck at 0 and the sampled value was 16'h7FFF

- **<u>Simple direct test bench for designer</u>**

```
module config_reg_direct_tb ();
  //? local parameters
  localparam CLK_CYCLE = 20;
  //? Interface signals
  bit clk;
  logic reset;
  logic write;
  logic [15:0] data_in;
  logic [15:0] data_out;
  typedef enum logic [2:0] {
    adc0_reg,
    adc1_reg,
    temp_sensor0_reg,
    temp_sensor1_reg,
    analog_test,
    digital_test,
    amp_gain,
    digital_config
  } config_reg_e;
  config_reg_e reg_addr;
  //? Variables
  int error_count, correct_count;
  logic [15:0] reset_assoc[config_reg_e];
  logic [15:0] golden_reg [config_reg_e];
  //? DUT instantiation
  config_reg DUT (
      .clk(clk),
      .reset(reset),
      .write(write),
      .data_in(data_in),
      .address(reg_addr),
      .data_out(data_out)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? assign default reset values
  initial begin
    reset_assoc[adc0_reg]         = 16'hFFFF;
    reset_assoc[adc1_reg]         = 0;
    reset_assoc[temp_sensor0_reg] = 0;
    reset_assoc[temp_sensor1_reg] = 0;
    reset_assoc[analog_test]      = 16'hABCD;
    reset_assoc[digital_test]     = 0;
    reset_assoc[amp_gain]         = 0;
    reset_assoc[digital_config]   = 1;
  end
  //? Driver block
  initial begin
    //* initialization
    reset = 0;
```

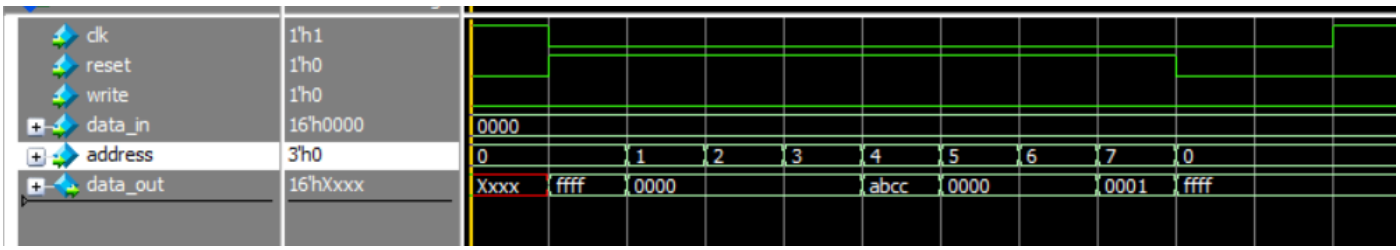
```
//? Driver block
initial begin
  //* initialization
  reset = 0;
  write = 0;
  data_in = 0;
  reg_addr = adc0_reg;
  #CLK_CYCLE;
  //! analog_test register reset bug
  $display("RESET PHASE");
  assert_reset();


  //! adc0_reg test bug most bit stucks at 1
  @(negedge clk);
  $display("adc0_reg BUG");
  write = 1;
  data_in = 0;
  reg_addr = adc0_reg;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
  check_result(reg_addr, golden_reg);


  //! adc1_reg test bug least 8-bits swapped with most 8-bits
  $display("adc1_reg BUG");
  data_in = 16'hFF00;
  reg_addr = adc1_reg;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
  check_result(reg_addr, golden_reg);


  //! temp_sensor0_reg test bug store data_in after shift left by 1-bit
  $display("temp_sensor0_reg BUG");
  data_in = 1;
  reg_addr = temp_sensor0_reg;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
  check_result(reg_addr, golden_reg);


  //! digital_test & amp_gain registers test bug each one stores the value that should be written to the other
  $display("digital_test BUG");
  data_in = 16'hAAAA;
  reg_addr = digital_test;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
```

```
  data_in = 16'hFFFF;
  reg_addr = amp_gain;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
  check_result(reg_addr, golden_reg);


  //! digital_config register test bug most bit stucks at 0
  $display("digital_config BUG");
  data_in = 16'hFFFF;
  reg_addr = digital_config;
  golden_reg[reg_addr] = data_in;
  @(negedge clk);
  check_result(reg_addr, golden_reg);

  $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
  $stop;
end
//? TASKS
task assert_reset();
  reset = 1;
  @(negedge clk);
  check_result(reg_addr, reset_assoc);
  reset = 0;
endtask

task automatic check_result(config_reg_e reg_address, ref logic [15:0] gold_reg[config_reg_e]);
  reg_addr = reg_addr.first;
  for (int i = 0; i < reg_addr.num; i++) begin
    #(CLK_CYCLE / 20);
    if (data_out == gold_reg[reg_addr]) correct_count++;
    else begin
      error_count++;
      $display("\tXXXX ERROR at time %0t: the actual output result is not matching the expected result XXXX",$time);
      $display("\taddr=%s(%0d) actual_data_out=%h expected_data_out=%h\n", reg_addr.name(), int'(reg_addr), data_out, gold_reg[reg_addr]);
    end
    golden_reg[reg_addr] = data_out;
    reg_addr = reg_addr.next;
  end
  reg_addr = reg_address;
endtask
```

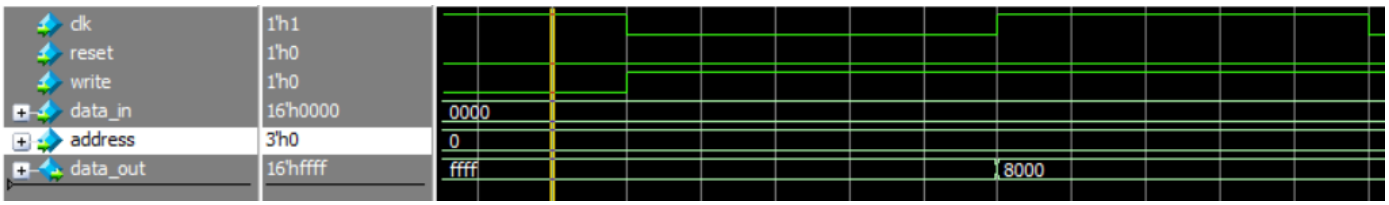
- **Simulation wave form results**

```
# vsim -voptargs="+acc" work.config_reg_direct_tb -coverage
# Start time: 13:46:41 on Apr 18,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
# Loading sv_std.std
# Loading work.config_reg_direct_tb(fast)
# Loading work.config_reg(fast)
# RESET PHASE
#       xxxx ERROR at time 25: the actual output result is not matching the expected result xxxx
#       addr=analog_test(4) actual_data_out=abcc expected_data_out=abcd
#
# adc0_reg BUG
#       xxxx ERROR at time 61: the actual output result is not matching the expected result xxxx
#       addr=adc0_reg(0) actual_data_out=8000 expected_data_out=0000
#
# adc1_reg BUG
#       xxxx ERROR at time 82: the actual output result is not matching the expected result xxxx
#       addr=adc1_reg(1) actual_data_out=00ff expected_data_out=ff00
#
# temp_sensor0_reg BUG
#       xxxx ERROR at time 103: the actual output result is not matching the expected result xxxx
#       addr=temp_sensor0_reg(2) actual_data_out=0002 expected_data_out=0001
#
# digital_test BUG
#       xxxx ERROR at time 146: the actual output result is not matching the expected result xxxx
#       addr=digital_test(5) actual_data_out=ffff expected_data_out=aaaa
#
#       xxxx ERROR at time 147: the actual output result is not matching the expected result xxxx
#       addr=amp_gain(6) actual_data_out=aaaa expected_data_out=ffff
#
# digital_config BUG
#       xxxx ERROR at time 168: the actual output result is not matching the expected result xxxx
#       addr=digital_config(7) actual_data_out=7fff expected_data_out=ffff
#
# error_counts = 7, correct_counts = 41
# ** Note: $stop    : config_reg_direct_tb.sv(111)
#    Time: 168 ns  Iteration: 0  Instance: /config_reg_direct_tb
```

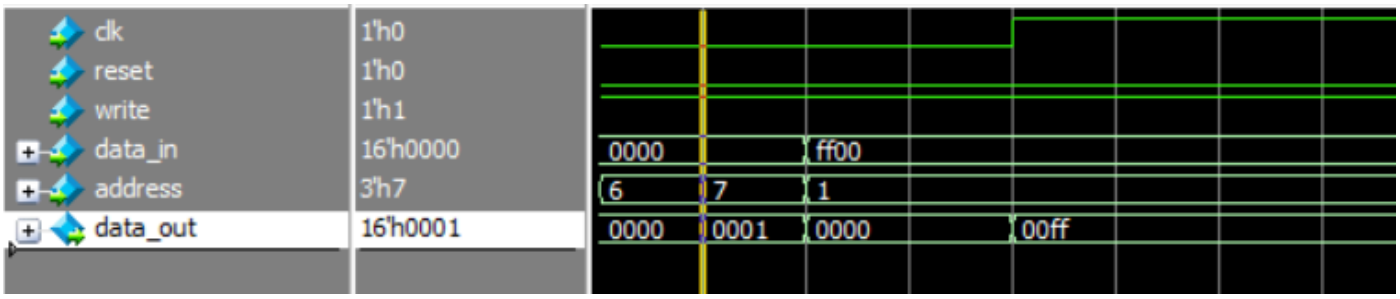
❖ **Reset**



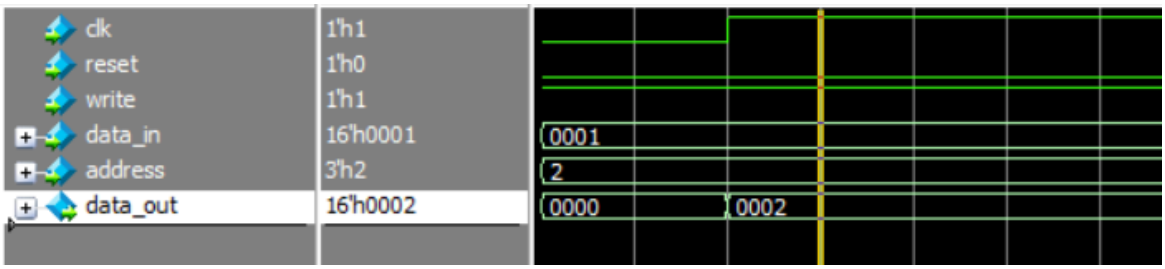
❖ **Adc0_reg bug**



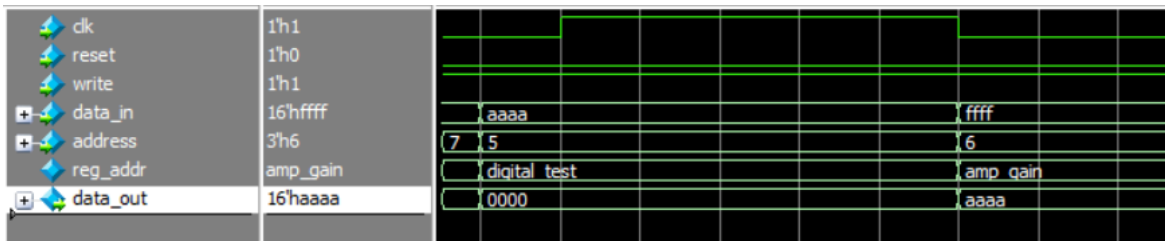
❖ **Adc1_reg bug**

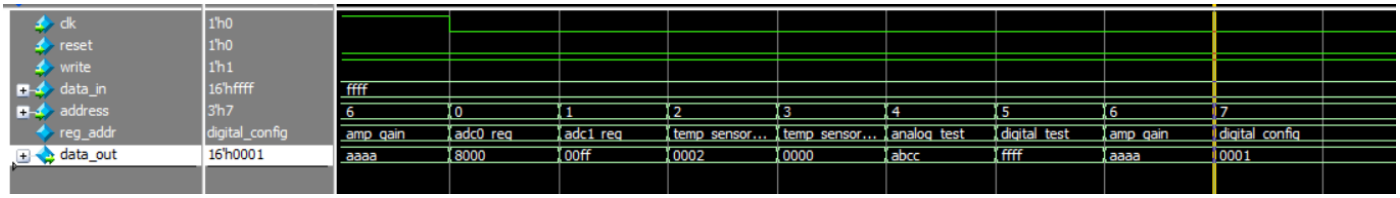
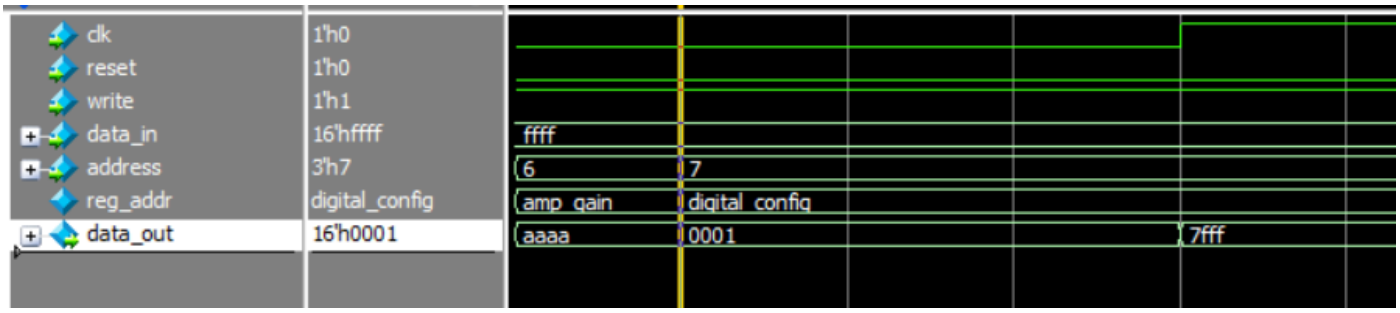


❖ **Temp_sensor0_reg bug**



❖ **Digital_test and amp_gain bug**





❖ **Config_reg bug**